面试准备包 · 本地优先的 AI 日程助手

一、二部分念到能自然说出来；三部分快速过；四部分只在卡壳时用。 原则：**不会就说不确定，但把边界说清楚**——比硬编更有说服力。

一、项目介绍（口述稿，约 2 分钟）

这是一个**本地优先的 AI 日程助手**：用户说”下周二上午九点加一门计算机原理在教三 201”， 系统转成结构化工具调用并落库；二阶段再进一步，用户贴一段杂乱待办， 系统抽成约束、用求解器排出一整周计划。

两个关键设计。第一，**模型只做语义归一化，不做计算**——契约里只允许”下周三” “下个月 3 号” 这类相对表达，绝对时间由确定性代码解析，彻底消掉”模型算错日期”这一整类错误。 第二，**规划交给求解器。**让 LLM 直接产出可行计划在公开基准上准确率只有三成量级， 所以我拆成”模型抽约束 + CP-SAT 求解”，求解只要 20–30ms，而且排不出来时能说清是哪几条约束在打架。

工程上最得意的是**一份 JSON Schema 驱动四个消费方**：约束解码语法、LoRA 训练标签、 执行层校验、MCP 工具定义，全部同源，CI 有断言。实测端到端中位 **641ms**——这台机器只有核显。

二、模拟 Q&A

2.1 基础部分

Q：解决什么问题？ A：把”人话”变成”可靠的工具调用”。难点不在理解中文，而在**输出必须 100% 合法、语义必须正确。** 做法是把不确定的部分（语言理解）和确定的部分（日期解析、排期）切开。

Q：为什么不用 GPT-4 / 云端 API？ A：**隐私**（日程是最私人的数据，不该出本机）、**成本**、**可用性**（断网也要能用）。 代价是小模型能力弱，所以要用工程手段补。

Q：什么是约束解码？ A：把 JSON Schema 编译成**文法（GBNF）**，推理时每个 token 只能在文法允许的集合里采样。 效果是输出**结构上不可能非法。**但它只保证格式，**不保证语义对。**

Q：为什么坚持"模型不算日期"？ A：模型做算术是**不可测的错**——错一周在界面上看不出来，用户不会报错、只会某天发现事没做。 移到确定性代码后这类 bug 消失，且能被单测覆盖。

Q：MCP 是什么？为什么用？ A：MCP 是**工具怎么被描述、发现、调用**的标准协议。它替代的是"工具层"， 不是"把中文变成 JSON"那一层——这两件事常被混为一谈。用它是因为客户端能零适配接入。

Q：CP-SAT 干什么？ A：解带约束的排期。输入待办（时长/截止/优先级）和限制（哪段不能排）， 输出满足全部硬约束、软约束代价最小的周计划。确定性求解器，可复现、可解释。

2.2 技术深入部分

Q：约束解码的代价？"保证格式不保证语义"具体指什么？ A：两个代价：**首 token 延迟**（语法预编译）和**表达受限**（Schema 没定义的出不来， 所以契约设计本身就是产品设计）。它能保证 `module` 在枚举里，但保证不了"开会"该归 `meeting` 还是 `class`。所以执行层留了 `clarify`（反问）和 `reject`（拒答）两个出口。

Q：4B 为什么反而更差？ A：Qwen3 默认开 thinking，4B 把 token 预算全花在推理链上——实测 298 个 reasoning chunk、 **正文一个字都没输出。**我试过 `--reasoning-budget 0`，**无效**：它只把文本挪到 `content`， 模型还在乱想，还给错了年份。真正有效的是 `--reasoning off`，输出**逐字一致**，1900ms → 641ms。

Q："一份 Schema 四处消费"怎么防漂移？ A：四处消费方式不同但来源相同：约束解码生成 GBNF；训练集生成器决定标签字段； 执行层用 Pydantic 镜像校验；MCP 服务端用 `Literal[*SCHEMA_ENUM]` 直接取枚举。 CI 两条断言：Pydantic 镜像字段集合 == Schema；MCP 工具枚举 == Schema 枚举。 "训练时一个字段、调用时另一个字段"这种静默失效，提交前就会被拦住。

Q：反向构造怎么保证标签对？怎么证明校验器有效？ A：**先有标签、再渲染文本**，标签从真值生成，构造上不会错。但渲染可能漏说—— 文本没说时长、标签却写 120 分钟，模型就学到瞎猜。所以另写校验器逐条断言 "标签每个字段都能从文本推断"，且**按分句**检查（全文检查会被别的事项的措辞打掩护）。 更关键的是给校验器写自测：故意改

坏五类字段，断言必须全抓到、干净数据 0 误报。 **一个永远通过的校验器等于没有校验器**——第一版我踩过：把 `priority` 改成 `high`， 但样本本来就是 `high`，等于没改。

Q：规则层和模型的边界怎么划？会不会改坏模型对的答案？ A：标准是”**规则做得到且更准的，就不给模型**”。模型抽 `keep_free`（哪段别排事）只有 30.6%， 而”周三上午别排东西”这种句式正则几乎不会错，接管后到 94.4%。 踩过两次坑：把约束里的”周三”当成事项截止日 → **先抽约束、记下字符区间、抹掉后再给 事项找日期**；标题与文本对不上时退化全文查找、抓到别的事项时长 → **定位不到就不改。**

Q：39% 怎么上到 80%？ A：短期靠微调（数据集与训练套件已就绪，4000 条反向构造、标签 100% 校验通过，只差 GPU）。 长期靠**联合分布**：基座逐 token 贪心，没有”整条样本要自洽”的概念，所以单字段都对、凑一起对不齐。 另外会做 `plan_diff` 局部重排——比整周重排容错性高，单次错误影响面更小。

三、技术知识点扫盲

3.1 基础

概念	一句话
LoRA	冻结原模型，只训一对低秩矩阵（秩 r 通常 16），显存成本降一两个数量级
GGUF / 量化	llama.cpp 的模型格式；Q4_K_M 是 4bit 混合量化，体积约 fp16 的 1/4
约束解码 / GBNF	按文法限制 token 候选集使输出结构合法；GBNF 是 llama.cpp 的文法语言
CP-SAT / NoOverlap	约束规划求解器，擅长排期；NoOverlap 约束一组区间互不重叠
MCP	工具/上下文暴露的标准协议，管”描述、发现、调用”

3.2 技术深入

概念	要点
thinking mode	Qwen3 默认先生成推理链；`--reasoning-`

概念	要点
	`budget 0` **不能**关闭，必须 `--reasoning off`
首 token 延迟 vs 总时长	约束解码拉高首字延迟（语法编译），总时长可能更短；评测要分开看
bf16 vs fp16	原生 bf16 需 sm_80+；`torch.cuda.is_bf16_supported()` 在 sm_75 上**误报 True**
逐 token 贪心 vs 联合自洽	单字段准确率相乘会急剧衰减，这是”每字段都对、整条不对”的根因
Wilson 置信区间	小样本下比正态近似可靠；21 条样本的 95% CI 有 ±11pp，不足以支撑结论
契约漂移	多处手写同一结构导致逐渐不一致；解法是单一事实源 + CI 断言

四、答不上来时的应对话术

目标不是糊弄，而是**展示你知道边界在哪、以及会怎么查。**

1. **完全没听过** → “这块我没深入了解，不敢乱说。我理解它在解决 X 类问题，但机制不确定。 如果要用，我会先看官方文档和最小可跑例子，再回来确认。”
2. **记得结论忘了推导** → “结论我记得是 X，但为什么得再推一遍。当时是实验得出的， 我可以把测法讲一下——你要结论还是方法？”
3. **被追问到盲区** → “这点我确实没考虑到。我猜影响可能在 A 和 B 两方面，但需要验证。 您能说说是哪个方向吗？我想顺着学一下。” （**把追问变成学习机会**）
4. **数据记不准** → “具体数字我不保证，量级是 600 多毫秒，准确值在仓库 benchmark 里。 我更确定的是**相对关系**：关掉 thinking 后快了约 3.6 倍，输出逐字一致。”
5. **被指出方案有问题** → “您说得对，这确实是漏洞。我当时在 X 和 Y 之间取了 X，原因是……。 如果按您说的改，我担心引入 Z。您觉得 Z 可以接受吗？” （**承认取舍，不硬辩**）

6. **通用兜底** → “总体我还不确定，但有一点肯定：_____。其余我想确认完再说。”

附：面试前 5 分钟速览

- 核心叙事：**模型不算日期 / 规划交给 CP-SAT / 一份 Schema 四处消费**
- 三个数字：**641ms**、**4372/4372**、**4%→39%**
- 三个诚实点：抽取准确率还不够、没做多轮对话、微调因缺 GPU 未跑
- 收尾：**“我最想展示的不是功能，是把自己’不确定的模型’包进’确定的工程’里的过程。”**